PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering
(AI & ML)
B. Tech. CSE - 5th Semester
Aug – Dec 2026

UE24CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

DebtLens – Global External Debt Risk and Creditor Analysis System

Submitted by:

PES1UG24AM233	Ryona Aditree Ghosh	5D
PES1UG24AM203	Preeti Kalburgi	5D

Class of Prof. ***Raghu B A***
Lab Faculty: Devna Sumesh

Abstract

DebtLens is a database system for organising and analysing the external debt of countries. PostgreSQL stores regions, countries, indicators, yearly debt observations, creditors, creditor allocations and risk assessments in seven related tables, and is queried through five analytical reports, three views, a stored function, indexes and tested transactions. MongoDB stores country metadata, raw World Bank API records and data import logs. A web interface with a login page, dashboard, country comparison, debt trend and risk analysis screens sits on top of the databases. The PostgreSQL demo data is synthetic and illustrative, and the risk scores are project-defined values, not official sovereign credit ratings.

Table of Contents

Sl. No	Topic	Page No.
1.	Introduction a) Problem Statement	3
2.	User Requirements Specification- Tech Stack	3
3.	E-R Model a) List of Entities [labeling Weak ones] b) List non-simple Attributes c) ER Diagram Snapshot d) ERD from tool like draw.io	4
4.	Relational Model a) List of Relations that are not Entities in ER Model b) Schema Diagram [ensure light background image]	5
5.	SQL DDL Statements	6
6.	SQL DML Statements	8
7.	NoSQL/ Vector Database Commands	11
8.	Results a) Resulting Tables' Screenshots	13
9.	Next.js-Project Tree Structure	16
10.	Few sample Screenshots of application code and demo	16
11.	Evaluation Rubrics- Completion Status	20

Introduction

Problem Statement

External debt figures are usually spread across many reports and spreadsheets, which makes it hard to compare countries, follow one country's debt over several years, or see which creditors it owes money to. DebtLens puts this information into databases so that country comparisons, regional summaries, historical trends and risk exploration can be answered with queries.

Short Description

DebtLens organises external debt information and supports country comparisons, regional summaries, historical trend analysis and project-defined debt-risk exploration. PostgreSQL stores the structured data (regions, countries, indicators, annual debt observations, creditors, creditor allocations and risk assessments), while MongoDB keeps the raw World Bank records, country metadata and import logs. The PostgreSQL demo dataset contains synthetic illustrative values; they are not verified World Bank statistics and the stored risk scores are not official sovereign credit ratings.

User Requirements Specification

Technology Stack

Layer	Technology	Used for
Relational database	PostgreSQL (psql)	Tables, constraints, indexes, views, stored function, transactions
NoSQL database	MongoDB	Country metadata, raw World Bank records, import logs, aggregation

Web application	Next.js	Login, dashboard, country comparison, trends and risk analysis
Data source	World Bank API	Country-level external debt indicators, 2015–2023

Why this stack

The core data is structured and connected: a country belongs to a region, a debt observation belongs to a country and an indicator, and creditors are linked to observations through allocations. PostgreSQL enforces these links with keys and constraints, and gives the joins, window functions, views and transactions the reports need. Next.js was chosen so the dashboard and report screens can read from the databases through API routes.

Choice of NoSQL database

The World Bank API returns JSON records. MongoDB stores these as documents exactly as they arrive, along with country metadata and a log of each import, without first forcing them into tables. Its aggregation pipeline ($group, $lookup and others) is then used to summarise the raw records.

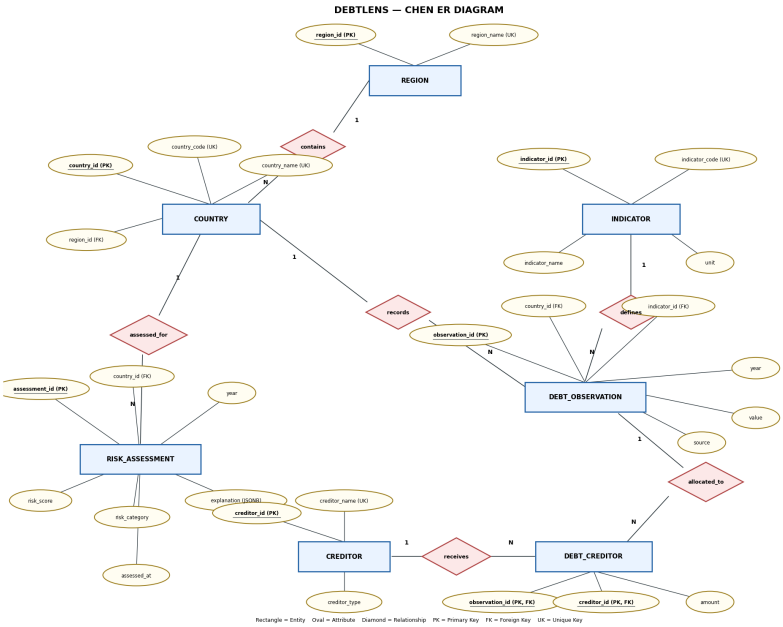
E-R Model

List of Entities

Entity	Primary key	Other attributes	Type
REGION	region_id	region_name (unique)	Strong
COUNTRY	country_id	country_code (unique), country_name (unique), region_id (FK)	Strong
INDICATOR	indicator_id	indicator_code (unique), indicator_name, unit	Strong
DEBT_OBSERVATION	observation_id	country_id (FK), indicator_id (FK), year, value, source	Strong
CREDITOR	creditor_id	creditor_name (unique), creditor_type	Strong
RISK_ASSESSMENT	assessment_id	country_id (FK), year, risk_score, risk_category, explanation, assessed_at	Strong
DEBT_CREDITOR	observation_id + creditor_id	amount	Junction

There are no weak entities. DEBT_CREDITOR only resolves the many-to-many link between DEBT_OBSERVATION and CREDITOR.

ER Diagram



Chen ER diagram of DebtLens

Non-simple Attributes

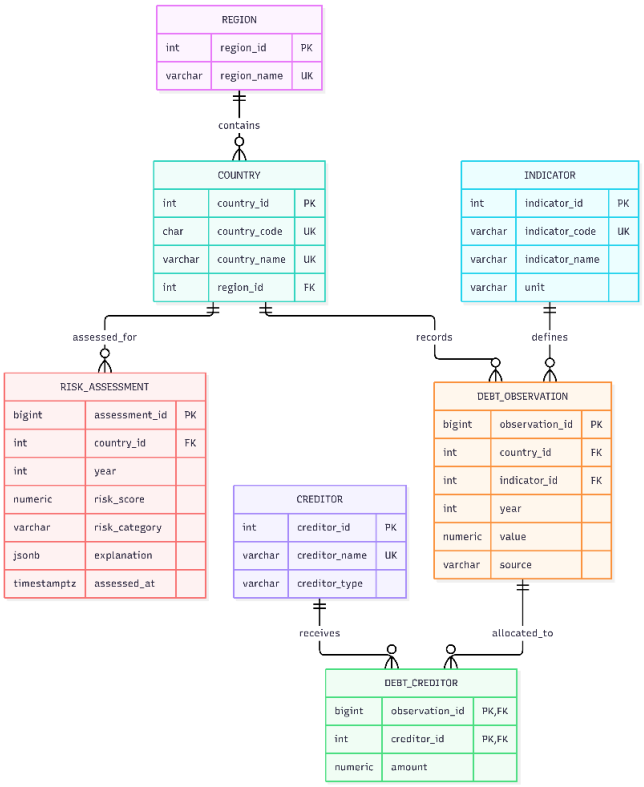
RISK_ASSESSMENT.explanation is a JSONB document that can hold several reasons for a score; all other attributes are simple. Debt change values are derived in the year-over-year report and not stored.

Relational Model

List of Relations

Six relations correspond directly to entities. One relation is not an entity in the ER model: DEBT_CREDITOR (observation_id, creditor_id, amount), created from the many-to-many relationship between DEBT_OBSERVATION and CREDITOR. Its primary key is (observation_id, creditor_id) and both columns are foreign keys.

Schema Diagram



Relational schema of DebtLens (data types, primary, foreign and unique keys)

SQL DDL Statements

The schema, constraints and indexes are defined in database/schema.sql. GitHub repository: https://github.com/Preeti1k3/DebtLens.git

```
-- DebtLens: Global External Debt Risk & Creditor Analysis System
-- Database: PostgreSQL

CREATE TABLE IF NOT EXISTS region (
   region_id SERIAL PRIMARY KEY,
   region_name VARCHAR(100) UNIQUE NOT NULL
);
CREATE TABLE

CREATE TABLE IF NOT EXISTS country (
   country_id SERIAL PRIMARY KEY,
   country_code CHAR(3) UNIQUE NOT NULL,
   country_name VARCHAR(150) UNIQUE NOT NULL,
   region_id INT REFERENCES region(region_id)
);
CREATE TABLE

CREATE TABLE IF NOT EXISTS indicator (
   indicator_id SERIAL PRIMARY KEY,
   indicator_code VARCHAR(100) UNIQUE NOT NULL,
   indicator_name VARCHAR(200) NOT NULL,
   unit VARCHAR(50) NOT NULL
);
CREATE TABLE

CREATE TABLE IF NOT EXISTS debt_observation (
   observation_id BIGSERIAL PRIMARY KEY,
   country_id INT NOT NULL
       REFERENCES country(country_id),
   indicator_id INT NOT NULL
       REFERENCES indicator(indicator_id),
   year INT NOT NULL CHECK (year BETWEEN 1960 AND 2100),
   value NUMERIC(20, 4) CHECK (value >= 0),
   source VARCHAR(100) NOT NULL DEFAULT 'World Bank',
   UNIQUE (country_id, indicator_id, year)
);
CREATE TABLE

CREATE TABLE IF NOT EXISTS creditor (
   creditor_id SERIAL PRIMARY KEY,
   creditor_name VARCHAR(150) UNIQUE NOT NULL,
   creditor_type VARCHAR(80) NOT NULL
);
CREATE TABLE

CREATE TABLE IF NOT EXISTS debt_creditor (
   observation_id BIGINT NOT NULL
```

```
        REFERENCES debt_observation(observation_id)
        ON DELETE CASCADE,
    creditor_id INT NOT NULL
        REFERENCES creditor(creditor_id),
    amount NUMERIC(20, 4) NOT NULL CHECK (amount >= 0),
    PRIMARY KEY (observation_id, creditor_id)
);
CREATE TABLE

CREATE TABLE IF NOT EXISTS risk_assessment (
    assessment_id BIGSERIAL PRIMARY KEY,
    country_id INT NOT NULL
        REFERENCES country(country_id),
    year INT NOT NULL CHECK (year BETWEEN 1960 AND 2100),
    risk_score NUMERIC(5, 2) NOT NULL
        CHECK (risk_score BETWEEN 0 AND 100),
    risk_category VARCHAR(20) NOT NULL
        CHECK (risk_category IN (
            'Low', 'Moderate', 'High', 'Very High'
        )),
    explanation JSONB NOT NULL DEFAULT '{}'::jsonb,
    assessed_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (country_id, year)
);
CREATE TABLE

-- Indexes for commonly queried columns
CREATE INDEX IF NOT EXISTS idx_debt_country_year
    ON debt_observation(country_id, year);
CREATE INDEX

CREATE INDEX IF NOT EXISTS idx_debt_indicator_year
    ON debt_observation(indicator_id, year);
CREATE INDEX

CREATE INDEX IF NOT EXISTS idx_risk_category_year
    ON risk_assessment(risk_category, year);
CREATE INDEX

CREATE INDEX IF NOT EXISTS idx_debt_creditor_creditor
    ON debt_creditor(creditor_id);
CREATE INDEX
```

```
debtlens=# \dt
                List of tables
 Schema |       Name        | Type  |  Owner
--------+-------------------+-------+----------
 public | country           | table | postgres
 public | creditor          | table | postgres
 public | debt_creditor     | table | postgres
 public | debt_observation  | table | postgres
 public | indicator         | table | postgres
 public | region            | table | postgres
 public | risk_assessment   | table | postgres
(7 rows)


debtlens=#
```

Table list in PostgreSQL (\dt)

```
debtlens=# \d country
                                    Table "public.country"
    Column    |         Type         | Collation | Nullable |                  Default
--------------+----------------------+-----------+----------+-------------------------------------------
 country_id   | integer              |           | not null | nextval('country_country_id_seq'::regclass)
 country_code | character(3)         |           | not null |
 country_name | character varying(150) |           | not null |
 region_id    | integer              |           |          |
Indexes:
    "country_pkey" PRIMARY KEY, btree (country_id)
    "country_country_code_key" UNIQUE CONSTRAINT, btree (country_code)
    "country_country_name_key" UNIQUE CONSTRAINT, btree (country_name)
Foreign-key constraints:
    "country_region_id_fkey" FOREIGN KEY (region_id) REFERENCES region(region_id)Referenced by:
    TABLE "debt_observation" CONSTRAINT "debt_observation_country_id_fkey" FOREIGN KEY (country_id) REFERENCES country(country_id)
    TABLE "risk_assessment" CONSTRAINT "risk_assessment_country_id_fkey" FOREIGN KEY (country_id) REFERENCES country(country_id)
```

Structure of the country table

```
debtlens=# \d debt_observation
                                        Table "public.debt_observation"
     Column     |          Type          | Collation | Nullable |                        Default
----------------+------------------------+-----------+----------+-------------------------------------------------------
 observation_id | bigint                 |           | not null | nextval('debt_observation_observation_id_seq'::regclass)
 country_id     | integer                |           | not null |
 indicator_id   | integer                |           | not null |
 year           | integer                |           | not null |
 value          | numeric(20,4)          |           |          |
 source         | character varying(100) |           |          | 'World Bank'::character varying
Indexes:
    "debt_observation_pkey" PRIMARY KEY, btree (observation_id)
    "debt_observation_country_id_indicator_id_year_key" UNIQUE CONSTRAINT, btree (country_id, indicator_id, year)
    "idx_debt_country_year" btree (country_id, year)
    "idx_debt_indicator_year" btree (indicator_id, year)
Check constraints:
    "debt_observation_value_check" CHECK (value >= 0::numeric)
    "debt_observation_year_check" CHECK (year >= 1960 AND year <= 2100)
Foreign-key constraints:
    "debt_observation_country_id_fkey" FOREIGN KEY (country_id) REFERENCES countr-- More  --
```

Structure of the debt_observation table

```
debtlens_test=# \di
                                 List of indexes
 Schema |                     Name                     | Type  |  Owner   |      Table
--------+----------------------------------------------+-------+----------+-----------------
 public | country_country_code_key                     | index | postgres | country
 public | country_country_name_key                     | index | postgres | country
 public | country_pkey                                 | index | postgres | country
 public | creditor_creditor_name_key                   | index | postgres | creditor
 public | creditor_pkey                                | index | postgres | creditor
 public | debt_creditor_pkey                           | index | postgres | debt_creditor
-- More  --
```

Index inventory (\di)

SQL DML Statements

Demo data is loaded from database/seed.sql and the reports are in database/reports.sql. The values are illustrative and are not official World Bank statistics. The output of each report is shown in the Results section.

```
-- DebtLens demo data
INSERT INTO region (region_name)
VALUES
   ('South Asia'),
   ('Latin America'),
   ('Europe and Central Asia')
ON CONFLICT (region_name) DO NOTHING;

INSERT INTO country (country_code, country_name, region_id)
SELECT v.country_code, v.country_name, r.region_id
FROM (VALUES
   ('IND', 'India', 'South Asia'),
   ('BGD', 'Bangladesh', 'South Asia'),
   ('BRA', 'Brazil', 'Latin America'),
   ('TUR', 'Turkiye', 'Europe and Central Asia')
) AS v(country_code, country_name, region_name)
JOIN region r ON r.region_name = v.region_name
ON CONFLICT (country_code) DO NOTHING;

INSERT INTO indicator (indicator_code, indicator_name, unit)
VALUES
   ('EXT_DEBT_USD', 'External debt stock', 'USD millions'),
   ('DEBT_GNI', 'External debt as percentage of GNI', 'Percent')
ON CONFLICT (indicator_code) DO NOTHING;

INSERT INTO creditor (creditor_name, creditor_type)
VALUES
   ('Multilateral creditors', 'Multilateral'),
   ('Bilateral creditors', 'Bilateral'),
   ('Private creditors', 'Private')
ON CONFLICT (creditor_name) DO NOTHING;

-- Illustrative external debt stock, USD millions
INSERT INTO debt_observation
   (country_id, indicator_id, year, value, source)
SELECT c.country_id, i.indicator_id, v.year, v.value,
      'Synthetic demo data'
FROM (VALUES
   ('IND', 2022, 620000.0::NUMERIC),
   ('IND', 2023, 650000.0::NUMERIC),
   ('BGD', 2022,  95000.0::NUMERIC),
   ('BGD', 2023, 100000.0::NUMERIC),
   ('BRA', 2022, 550000.0::NUMERIC),
   ('BRA', 2023, 580000.0::NUMERIC),
   ('TUR', 2022, 450000.0::NUMERIC),
   ('TUR', 2023, 470000.0::NUMERIC)
) AS v(country_code, year, value)
JOIN country c ON c.country_code = v.country_code
JOIN indicator i ON i.indicator_code = 'EXT_DEBT_USD'
ON CONFLICT (country_id, indicator_id, year) DO NOTHING;

-- Illustrative external debt/GNI percentages
INSERT INTO debt_observation
   (country_id, indicator_id, year, value, source)
SELECT c.country_id, i.indicator_id, v.year, v.value,
      'Synthetic demo data'
FROM (VALUES
   ('IND', 2022, 18.0::NUMERIC),
   ('IND', 2023, 19.0::NUMERIC),
   ('BGD', 2022, 21.0::NUMERIC),
   ('BGD', 2023, 22.0::NUMERIC),
   ('BRA', 2022, 30.0::NUMERIC),
   ('BRA', 2023, 31.0::NUMERIC),
   ('TUR', 2022, 45.0::NUMERIC),
   ('TUR', 2023, 47.0::NUMERIC)
) AS v(country_code, year, value)
JOIN country c ON c.country_code = v.country_code
JOIN indicator i ON i.indicator_code = 'DEBT_GNI'
```

```
ON CONFLICT (country_id, indicator_id, year) DO NOTHING;

INSERT INTO risk_assessment
   (country_id, year, risk_score, risk_category, explanation)
SELECT c.country_id, v.year, v.risk_score, v.risk_category,
      jsonb_build_object(
          'basis', 'Illustrative demonstration only',
          'method', 'Project-defined example score'
      )
FROM (VALUES
   ('IND', 2023, 25.0::NUMERIC, 'Low'),
   ('BGD', 2023, 45.0::NUMERIC, 'Moderate'),
   ('BRA', 2023, 55.0::NUMERIC, 'High'),
   ('TUR', 2023, 75.0::NUMERIC, 'Very High')
) AS v(country_code, year, risk_score, risk_category)
JOIN country c ON c.country_code = v.country_code
ON CONFLICT (country_id, year) DO NOTHING;

-- REPORT 1: External debt stock by country for 2023
SELECT c.country_name, d.year, d.value AS external_debt_usd_millions
FROM debt_observation d
JOIN country c ON c.country_id = d.country_id
JOIN indicator i ON i.indicator_id = d.indicator_id
WHERE i.indicator_code = 'EXT_DEBT_USD'
 AND d.year = 2023
ORDER BY d.value DESC;

-- REPORT 2: Average external debt by region for 2023
SELECT r.region_name,
      ROUND(AVG(d.value), 2) AS average_debt_usd_millions,
      COUNT(DISTINCT c.country_id) AS country_count
FROM debt_observation d
JOIN country c ON c.country_id = d.country_id
JOIN region r ON r.region_id = c.region_id
JOIN indicator i ON i.indicator_id = d.indicator_id
WHERE i.indicator_code = 'EXT_DEBT_USD'
 AND d.year = 2023
GROUP BY r.region_name
ORDER BY average_debt_usd_millions DESC;

-- REPORT 3: Year-over-year debt change using the LAG window function
WITH annual_debt AS (
   SELECT c.country_name, d.year, d.value,
          LAG(d.value) OVER (
              PARTITION BY c.country_id
              ORDER BY d.year
          ) AS previous_year_debt
   FROM debt_observation d
   JOIN country c ON c.country_id = d.country_id
   JOIN indicator i ON i.indicator_id = d.indicator_id
   WHERE i.indicator_code = 'EXT_DEBT_USD'
)
SELECT country_name, year, value AS current_debt,
      previous_year_debt,
      ROUND(value - previous_year_debt, 2) AS absolute_change,
      ROUND(
          100.0 * (value - previous_year_debt)
          / NULLIF(previous_year_debt, 0), 2
      ) AS percentage_change
FROM annual_debt
ORDER BY country_name, year;

-- REPORT 4: Risk assessment summary
SELECT c.country_name, ra.year, ra.risk_score,
      ra.risk_category, ra.explanation
FROM risk_assessment ra
JOIN country c ON c.country_id = ra.country_id
WHERE ra.year = 2023
ORDER BY ra.risk_score DESC;

-- REPORT 5: Countries with 2023 debt above the overall average
SELECT c.country_name, d.value AS external_debt_usd_millions
FROM debt_observation d
JOIN country c ON c.country_id = d.country_id
JOIN indicator i ON i.indicator_id = d.indicator_id
WHERE i.indicator_code = 'EXT_DEBT_USD'
 AND d.year = 2023
 AND d.value > (
     SELECT AVG(d2.value)
     FROM debt_observation d2
     JOIN indicator i2 ON i2.indicator_id = d2.indicator_id
     WHERE i2.indicator_code = 'EXT_DEBT_USD'
       AND d2.year = 2023
 )
ORDER BY d.value DESC;
```

```
debtlens_test=# SELECT indicator_id, indicator_code, indicator_name
debtlens_test-# FROM indicator;
 indicator_id | indicator_code |           indicator_name
--------------+----------------+-----------------------------------
            1 | EXT_DEBT_USD   | External debt stock
            2 | DEBT_GNI       | External debt as percentage of GNI
(2 rows)
```

Indicators stored in the database

Transaction behaviour is in database/transactions.sql. The COMMIT test inserted a demonstration region and confirmed that it stayed after the transaction; the ROLLBACK test inserted a different region, rolled back, and confirmed that no matching row remained.

```
debtlens_test=*# INSERT INTO region (region_name)
debtlens_test-*# VALUES ('Transaction Demo Region');
INSERT 0 1
debtlens_test=*#
debtlens_test=*# COMMIT;
COMMIT
debtlens_test=# SELECT *
debtlens_test-# FROM region
debtlens_test-# WHERE region_name = 'Transaction Demo Region';
 region_id |       region_name
-----------+-------------------------
         4 | Transaction Demo Region
(1 row)


debtlens_test=#
```

COMMIT test

```
debtlens_test=# BEGIN;
BEGIN
debtlens_test=*# INSERT INTO region (region_name)
debtlens_test-*# VALUES ('Rollback Demo Region');
INSERT 0 1
debtlens_test=*# ROLLBACK;
ROLLBACK
debtlens_test=# SELECT *
debtlens_test-# FROM region
debtlens_test-# WHERE region_name = 'Rollback Demo Region';
 region_id | region_name
-----------+-------------
(0 rows)


debtlens_test=#
```

ROLLBACK test

NoSQL/ Vector Database Commands

MongoDB is the NoSQL database in DebtLens. It stores country metadata, raw World Bank records and data import logs.

Database Connection

```
const client = new MongoClient(process.env.MONGO_URI);
await client.connect();
const db = client.db(process.env.MONGO_DB);
```

Document Insertion and Deletion

Existing records are cleared before data is reloaded, and country metadata and raw debt records are inserted into their collections.

```
await db.collection('countries_meta').deleteMany({});
await db.collection('countries_meta').insertMany(docs);

await db.collection('raw_records').deleteMany({
   '_import.indicator': ind
});

await db.collection('raw_records').insertMany(
   rows.map(r => ({
       ...r,
       _import: {
           indicator: ind,
           fetchedAt: new Date()
       }
   }))
);
```

Import Logging

The import_logs collection records the source, indicator, start and finish time, record count and status of every import.

```
await db.collection('import_logs').insertOne({
```

```
   source,
   indicator: ind,
   startedAt,
   finishedAt,
   recordCount,
   status
});
```

Document Retrieval

These queries fetch country metadata without the aggregate regions, and the raw debt records that have a value for the selected indicator.

```
await db.collection('countries_meta')
   .find({ 'region.value': { $ne: 'Aggregates' } })
   .toArray();

await db.collection('raw_records')
   .find({
       'indicator.id': ind.wb,
       value: { $ne: null }
   })
   .toArray();
```

Aggregation Operators

Operator	Purpose
$match	Filters documents based on specified conditions
$group	Groups documents by a specified field
$avg	Calculates the average value
$sum	Calculates the sum of values
$first	Returns the first value in a group
$sort	Sorts documents
$limit	Restricts the number of returned documents
$project	Selects or transforms document fields
$round	Rounds numerical values
$lookup	Joins data from another collection
$unwind	Splits an array field into separate documents

World Bank API Integration

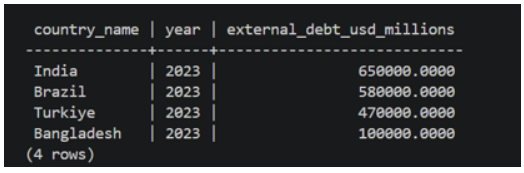
The World Bank API is the data source for country-level economic and external debt indicators. The endpoint is paginated and returns data for 2015–2023.

```
https://api.worldbank.org/v2/country/all/indicator/{INDICATOR}?format=json&per_page=1000&date=2015:2023&page={n}
```

Results

Resulting Tables' Screenshots

Output of the five analytical reports, which use joins, aggregation, grouping, sorting, subqueries, a CTE and the LAG() window function.



```
 country_name | year | external_debt_usd_millions
--------------+------+---------------------------
 India        | 2023 |                650000.0000
 Brazil       | 2023 |                580000.0000
 Turkiye      | 2023 |                470000.0000
 Bangladesh   | 2023 |                100000.0000
(4 rows)
```

Report 1: external debt by country

```
       region_name       | average_debt_usd_millions | country_count
-------------------------+---------------------------+---------------
 Latin America           |                580000.00 |             1
 Europe and Central Asia |                470000.00 |             1
 South Asia              |                375000.00 |             2
(3 rows)
```

Report 2: average debt by region

```
  country_name | year | current_debt | previous_year_debt | absolute_change | percentage_change
--------------+------+--------------+--------------------+-----------------+-------------------
 Bangladesh   | 2022 |   95000.0000 |                    |                 |
 Bangladesh   | 2023 |  100000.0000 |         95000.0000 |         5000.00 |              5.26
 Brazil       | 2022 |  550000.0000 |                    |                 |
 Brazil       | 2023 |  580000.0000 |        550000.0000 |        30000.00 |              5.45
 India        | 2022 |  620000.0000 |                    |                 |
 India        | 2023 |  650000.0000 |        620000.0000 |        30000.00 |              4.84
 Turkiye      | 2022 |  450000.0000 |                    |                 |
 Turkiye      | 2023 |  470000.0000 |        450000.0000 |        20000.00 |              4.44
(8 rows)
```

Report 3: year-over-year debt change

```
 country_name | year | risk_score | risk_category |                                                                                                explanation
--------------+------+------------+---------------+-----------------------------------------------------------------------------------------------------------------------------------------
 Turkiye      | 2023 |      75.00 | Very High     | {"basis": "Illustrative demonstration only", "method": "Project-defined example score"}
 Brazil       | 2023 |      55.00 | High          | {"basis": "Illustrative demonstration only", "method": "Project-defined example score"}
 Bangladesh   | 2023 |      45.00 | Moderate      | {"basis": "Illustrative demonstration only", "method": "Project-defined example score"}
 India        | 2023 |      25.00 | Low           | {"basis": "Illustrative demonstration only", "method": "Project-defined example score"}
(4 rows)
```

Report 4: risk assessment summary

```
  country_name | external_debt_usd_millions
--------------+----------------------------
 India        |                650000.0000
 Brazil       |                580000.0000
 Turkiye      |                470000.0000
(3 rows)
```

Report 5: countries above average debt

Three views were created in database/views.sql: vw_country_external_debt, vw_country_risk_summary and vw_debt_growth.

```
debtlens_test=# SELECT *
debtlens_test-# FROM vw_country_external_debt
debtlens_test-# WHERE year = 2023
debtlens_test-# ORDER BY external_debt_usd_millions DESC;
 country_name |       region_name       | year | external_debt_usd_millions
--------------+-------------------------+------+----------------------------
 India        | South Asia              | 2023 |                650000.0000
 Brazil       | Latin America           | 2023 |                580000.0000
 Turkiye      | Europe and Central Asia | 2023 |                470000.0000
 Bangladesh   | South Asia              | 2023 |                100000.0000
(4 rows)
```

vw_country_external_debt

```
debtlens_test=# SELECT country_name, year, risk_score, risk_category
debtlens_test-# FROM vw_country_risk_summary
debtlens_test-# WHERE year = 2023
debtlens_test-# ORDER BY risk_score DESC;
 country_name | year | risk_score | risk_category
--------------+------+------------+---------------
 Turkiye      | 2023 |      75.00 | Very High
 Brazil       | 2023 |      55.00 | High
 Bangladesh   | 2023 |      45.00 | Moderate
 India        | 2023 |      25.00 | Low
(4 rows)
```

vw_country_risk_summary

```
debtlens_test=# SELECT *
debtlens_test-# FROM vw_debt_growth
debtlens_test-# WHERE year = 2023
debtlens_test-# ORDER BY country_name;
 country_name | year | current_debt | previous_year_debt | absolute_change | percentage_change
--------------+------+--------------+--------------------+-----------------+-------------------
 Bangladesh   | 2023 |  100000.0000 |         95000.0000 |         5000.00 |              5.26
 Brazil       | 2023 |  580000.0000 |        550000.0000 |        30000.00 |              5.45
 India        | 2023 |  650000.0000 |        620000.0000 |        30000.00 |              4.84
 Turkiye      | 2023 |  470000.0000 |        450000.0000 |        20000.00 |              4.44
(4 rows)
```

vw_debt_growth

The function get_country_external_debt(p_country_code, p_year) returns the debt observation for a country code and year. Tests for India and Bangladesh returned their sample values, and a country missing from the dataset returned NULL.

```
debtlens_test=# \df get_country_external_debt
                                        List of functions
 Schema |          Name            | Result data type |          Argument data types           | Type
--------+--------------------------+------------------+----------------------------------------+------
 public | get_country_external_debt | numeric          | p_country_code character, p_year integer | func
(1 row)
```

Stored function definition

```
debtlens_test=# SELECT get_country_external_debt('IND', 2023);
 get_country_external_debt
---------------------------
                650000.0000
(1 row)


debtlens_test=#
```

Function test for India

```
debtlens_test=# SELECT get_country_external_debt('BGD', 2023);
 get_country_external_debt
---------------------------
                100000.0000
(1 row)
```

Function test for Bangladesh

```
debtlens_test=# SELECT get_country_external_debt('USA', 2023);
 get_country_external_debt
---------------------------

(1 row)


debtlens_test=#
```

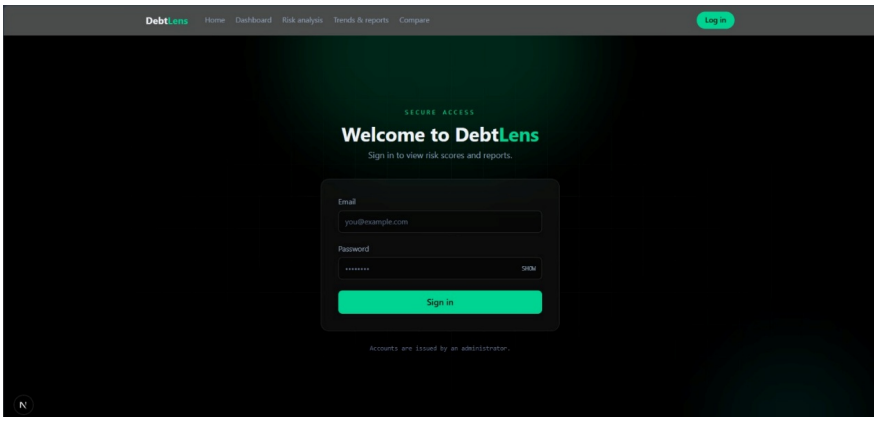
Function test for a country not in the dataset

Next.js - Project Tree Structure

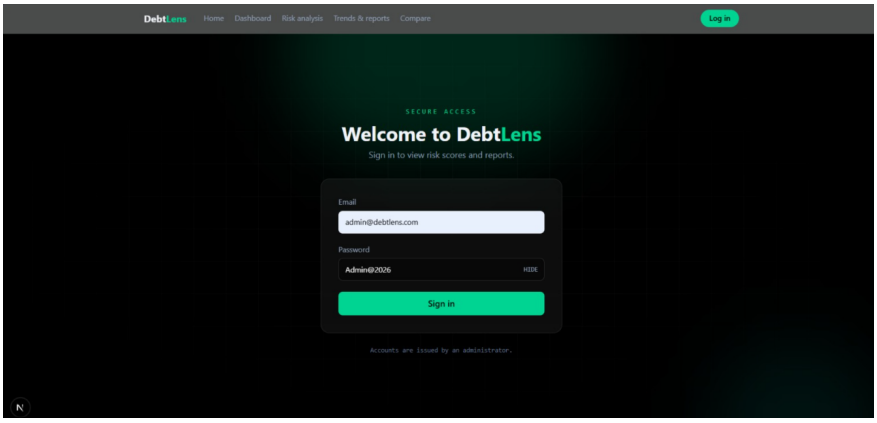
```
web
├── (
├── AGENTS.md
├── README.md
├── aggregation_output.txt
├── app
│   ├── admin
│   │   ├── actions.ts
│   │   └── page.tsx
│   ├── api
│   │   └── top-debt
│   │       └── route.ts
│   ├── compare
│   │   └── page.tsx
│   ├── country
│   │   └── [code]
│   │       └── page.tsx
│   ├── dashboard
│   │   └── page.tsx
│   ├── favicon.ico
│   ├── global.css
│   ├── global.tsx
│   ├── globals.css
│   ├── layout.tsx
│   ├── login
│   │   ├── actions.ts
│   │   └── page.tsx
│   ├── page.tsx
│   ├── reports
│   │   └── page.tsx
│   └── risk
│       └── page.tsx
├── components
│   ├── FinanceBackdrop.tsx
│   ├── LoginForm.tsx
│   ├── Nav.tsx
│   ├── Ticker.tsx
│   ├── charts.tsx
│   ├── country-charts.tsx
│   └── report-charts.tsx
├── eslint.config.mjs
├── lib
│   ├── auth.ts
│   ├── compareQueries.ts
│   ├── countryQueries.ts
│   ├── db.ts
│   ├── queries.ts
│   ├── reportQueries.ts
│   ├── risk
│   │   ├── config.ts
│   │   ├── riskScore.test.ts
│   │   └── riskScore.ts
│   ├── riskQueries.ts
│   └── token.ts
├── next-env.d.ts
├── next.config.ts
├── package-lock.json
├── package.json
├── proxy.ts
└── public
    ├── file.svg
    ├── globe.svg
    ├── next.svg
    ├── vercel.svg
    └── window.svg
```

```
├── scripts
│   ├── aggregations.mjs
│   ├── computeRisk.ts
│   ├── fixlinks.mjs
│   ├── ingest.mjs
│   ├── loadToPostgres.mjs
│   └── setupAuth.ts
├── tree.txt
└── tsconfig.json
```

Screenshots of Application Code and Demo



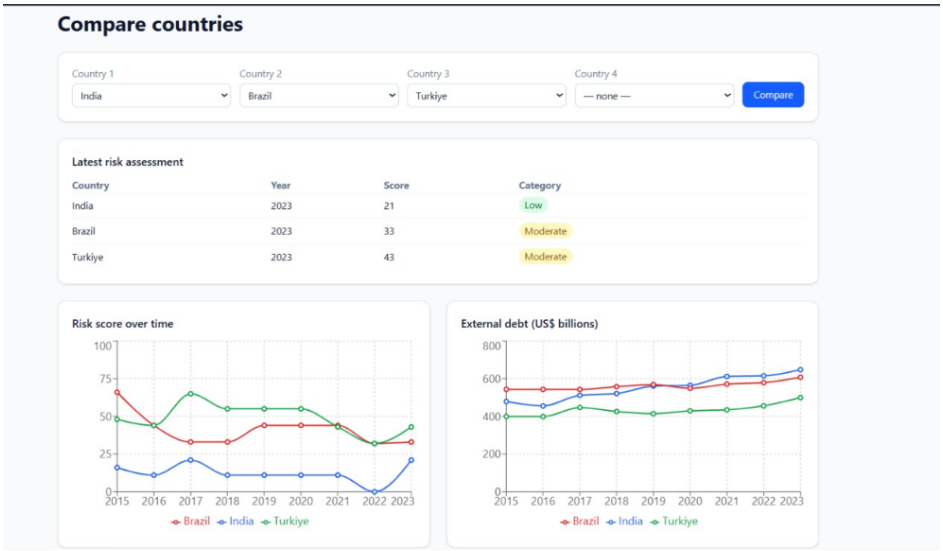
Login page



Login page



Main dashboard

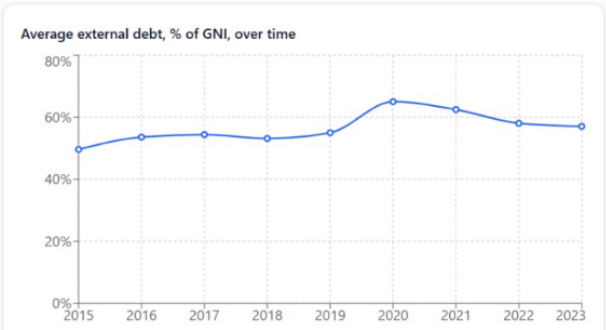


Country details and comparison

Top 15 countries by external debt

#	Country	Region	Debt (US$ millions)
1	China	East Asia & Pacific	2,447,537
2	India	South Asia	647,556
3	Brazil	Latin America & Caribbean	607,387
4	Mexico	Latin America & Caribbean	595,981
5	Turkiye	Europe & Central Asia	499,814
6	Indonesia	East Asia & Pacific	399,613
7	Argentina	Latin America & Caribbean	252,794
8	Colombia	Latin America & Caribbean	196,166
9	Thailand	East Asia & Pacific	194,693
10	Ukraine	Europe & Central Asia	178,989
11	Kazakhstan	Europe & Central Asia	171,484
12	Egypt, Arab Rep.	Middle East, North Africa, Afghanistan & Pakistan	168,182
13	South Africa	Sub-Saharan Africa	165,839
14	Viet Nam	East Asia & Pacific	141,845
15	Pakistan	Middle East, North Africa, Afghanistan & Pakistan	131,021

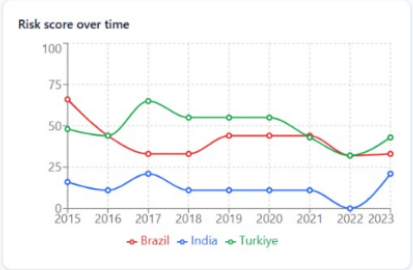
Country details and comparison (continued)

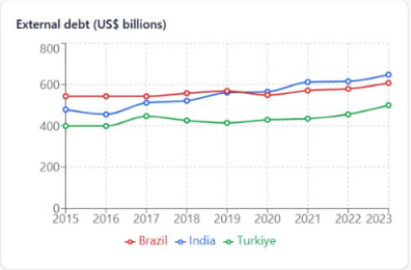


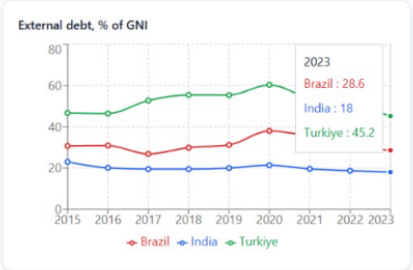
Highest-risk countries

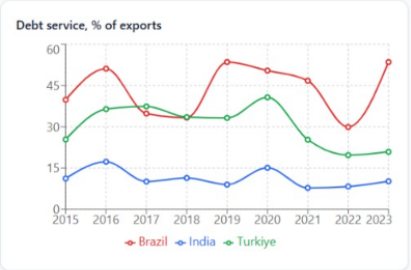
Country	Region	Score	Category
Senegal	Sub-Saharan Africa	88	Very High
Jamaica	Latin America & Caribbean	67	High
Lebanon	Middle East, North Africa, Afghanistan & Pakistan	67	High
Mongolia	East Asia & Pacific	67	High
Mozambique	Sub-Saharan Africa	67	High

Debt trends and reports

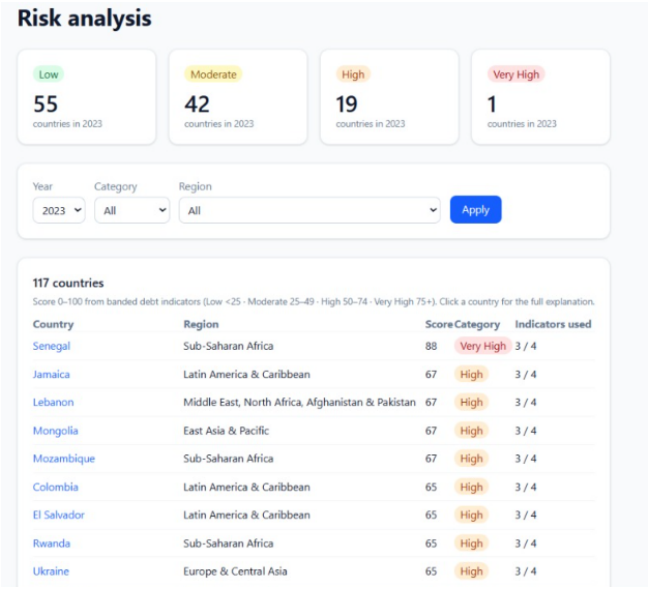








Debt trends and reports (continued)



Risk analysis

Evaluation Rubrics

#	Criteria	Distribution	Status
1	Database Design	ER diagram , relational schema with keys, relationships, and constraints	Completed
2	Database Creation	SQL database implementation - DDL with constraints , NoSQL/Vector database creation , Technology justification	Completed (PostgreSQL and MongoDB)
3	SQL Operations and Queries	DML operations , Nested/correlated/Recursive SQL queries, Join SQL queries	Completed (joins, subquery, CTE with LAG); no recursive query
4	Advanced Database Concepts	Advanced database concepts (Procedure/function, trigger, CTE) , NoSQL/Vector feature RUD	Function, views, indexes and CTE done; trigger not included; MongoDB update not shown
5	Web Application and Database Integration	Next.js interface , Backend/API and database integration	Completed (6 pages, node-postgres, MongoDB)
6	Authentication and Authorization	Authentication implementation (front end and backend) and minimum two roles and role-based access control in database	Completed in app; DB role grants pending
7	Documentation, Git/GitHub, Demonstration, Testing, and Originality	Documentation, Git/GitHub , Originality and individual contribution	Preeti: RDBMS part (schema, SQL, reports, views, function). Ryona: web app, risk scoring, auth. README, API tests and screenshots in progress.

GitHub repository: https://github.com/Preeti1k3/DebtLens.git